rPPG 技術、演算法與系統運作說明

產生日期：2026-05-21

本文件整理目前 mediapipe_rppg_system_v2 專案中 rPPG
模組實際使用到的技術、演算法、模型、資料流、計算方式與系統運作邏輯。內容以現有程式碼為準，主要來源為 rppg/rppg_worker.py、rppg/demo_rppg_live.py、gui/camera_stream_worker.py、gui/combined_worker.py、gui/
與 gui/metrics_recorder.py。

1. 系統定位

rPPG（remote
photoplethysmography，遠端光體積變化描記）模組的目標，是從一般 RGB
攝影機影像中擷取臉部皮膚區域的微弱顏色變化，估算脈搏波形，再推導心率、HRV
與壓力相關指標。本專案把 rPPG
作為「生理壓力／HRV」支柱，並可與行為疲勞偵測支柱融合成綜合疲勞分數。

目前 rPPG 模組輸出：

- BPM：每分鐘心跳數。
- RMSSD：相鄰 IBI 差值平方平均根，作為短期 HRV 指標。
- SDNN：IBI 標準差。
- LF/HF：心搏間期 tachogram 的低頻／高頻功率比。
- SQI：訊號品質指標。
- stress_prob 與 stress_status：以 LF/HF 與 RMSSD
推估的壓力機率與文字狀態。
- hrv_confidence：LOW、MEDIUM、HIGH。
- ibi_artifact_ratio：IBI 雜訊或異常比例。
- peak_freq_hz：Welch PSD 中主峰頻率。
- waveform、psd_freqs、psd_vals：供 GUI 與 Web
dashboard 顯示的波形與頻譜資料。

2. 使用技術與套件

2.1 OpenCV

OpenCV 負責攝影機開啟、影像讀取、色彩空間轉換、ROI mask、模糊、平均色彩與畫面標註。

實際用法：

- cv2.VideoCapture 開啟攝影機。
- Windows 優先使用 MSMF，可保留 MJPG 等原生格式；DirectShow
主要用於裝置名稱列舉。
- Linux 優先使用 V4L2。
- cv2.cvtColor 在 BGR 與 RGB 間轉換，以符合 MediaPipe 的
SRGB 輸入。
- cv2.fillConvexPoly 建立臉部 ROI 多邊形 mask。
- cv2.GaussianBlur 對 ROI 取樣前做 5x5 平滑。

- cv2.mean 搭配 mask 取得 ROI 平均 BGR/RGB 值。
- cv2.polylines 在預覽影像上畫出 ROI 邊界。

2.2 MediaPipe Face Landmarker

MediaPipe Face Landmarker 負責臉部 landmark 偵測。本專案使用 face_landmarker.task 模型，路徑為 models/face_landmarker.task。若模型不存在或大小異常，程式會從 Google MediaPipe model storage 下載：

https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/float16/latest/face_landmarker.task

實際設定：

- num_faces=1：只取一張臉，降低計算量並避免多臉混合。
- running_mode=VIDEO：逐幀處理影片串流，並傳入單調遞增 timestamp。
- output_face_blendshapes=True：目前已啟用，供眼、眉、嘴的輔助判定。
- output_facial_transformation_matrixes=True：目前已啟用，臉部姿態矩陣已用於 ROI roll 補償、姿態品質評分與 ROI 動態權重。
- min_face_detection_confidence=0.55
- min_face_presence_confidence=0.55
- min_tracking_confidence=0.60

VIDEO mode 要求每次呼叫 detect_for_video(mp_img, ts_ms) 時 timestamp 必須單調遞增。程式以第一幀時間作原點，並用 max(ts_ms_prev + 1, int((ts - t_origin) * 1000)) 保證 timestamp 不倒退。

2.2.1 Blendshape 輔助特徵

Face Landmarker 的 52 個 blendshape 係數現在被用作輔助訊號：

- eyeBlinkLeft、eyeBlinkRight：輔助眨眼與眼瞼閉合判定。
- eyeSquintLeft、eyeSquintRight：輔助瞇眼與半閉眼判定。
- browDownLeft、browDownRight、browInnerUp：輔助皺眉與壓力判定。
- jawOpen、mouthStretchLeft、mouthStretchRight：輔助哈欠與嘴部張開判定。

原則上，幾何 landmark 仍是主訊號，blendshape 只是增加穩定度與降低誤判。

2.3 NumPy

NumPy 負責時間序列陣列化、平均、標準差、clip、percentile、sliding window、頻譜資料遮罩與輸出轉換。

2.4 SciPy Signal

SciPy 是 rPPG 訊號處理核心：

- butter：建立 Butterworth band-pass filter。

- sosfiltfilt：以 second-order sections 做零相位濾波。
- filtfilt：當 SOS 路徑失敗時的備援零相位濾波。
- welch：估算 power spectral density。
- find_peaks：從濾波後波形找峰值。
- interp1d：將不等間隔 IBI tachogram 內插到固定 4 Hz，用於 LF/HF。

2.5 PySide6 與 Qt thread

GUI 中 rPPG 運算跑在 QThread，避免阻塞主視窗。主要 worker 為：

- CameraStreamWorker：讀取攝影機並送出 QImage 與 BGR frame queue。
- RppgWorker：單獨 rPPG 模式。
- CombinedWorker：rPPG 與行為疲勞同時啟用時，使用單一 MediaPipe Face Landmarker 推論，避免重複跑模型。

2.6 Flask / Socket.IO / SQLite

rPPG 指標會送到 Web dashboard 顯示，也可錄製到 SQLite：

- Web metrics 欄位包含 stress_bpm、stress_rmssd、stress_sdnn、stress_lf_hf、stress_sqi、stress_peak_freq、stress_prob、stress_wa 等。
- SQLite metrics 表保存 BPM、HRV、SQI、壓力、融合分數與相機資訊。

3. 系統資料流

GUI 啟動攝影機後，資料流如下：

- CameraStreamWorker 用 OpenCV 從指定攝影機讀取 BGR frame。
- 同一幀影像會被轉成 QImage 顯示，也會以 (timestamp, frame.copy()) 非阻塞送入 rPPG queue。
- 若只啟用 rPPG，RppgWorker 從 queue 取 frame。
- 若 rPPG 與行為疲勞都啟用，CombinedWorker 從 queue 取 frame，並以單一 Face Landmarker 同時供兩個引擎使用。
- worker 將 BGR 轉 RGB，建立 mp.Image。
- Face Landmarker 回傳 478 點臉部 landmark。
- 程式依 landmark 定義四個 ROI：額頭、左臉頰、右臉頰、眉間。
- 每個 ROI 計算 RGB 平均值；多個 ROI 依姿態動態權重加權成單筆 (R, G, B) 時間序列樣本。
- RppgEngine 保存最近約 12 秒樣本，估算取樣頻率。
- POS 演算法產生原始 pulse。
- Butterworth band-pass 過濾保留心率頻段。
- Welch PSD 取主峰換算 BPM。
- 波峰偵測產生 IBI，進一步計算 RMSSD、SDNN、LF/HF。

- SQI、artifact ratio 與 HRV confidence 評估結果可信度。
- 壓力 heuristic 產生 stress_prob 與 stress_status。
- 指標更新 GUI overlay、Web dashboard 與錄製資料庫。

4. 臉部 ROI 設計

本專案使用 MediaPipe 478 點 face mesh 中的四個皮膚區域：

- forehead：[107, 66, 69, 109, 10, 338, 299, 296, 336, 9]
- L_cheek：[118, 119, 100, 126, 209, 49, 129, 203, 205, 50]
- R_cheek：[347, 348, 329, 355, 429, 279, 358, 423, 425, 280]
- glabella：[151, 108, 107, 55, 8, 285, 336, 337]

每個 ROI 的處理方式：

- 將 landmark 正規化座標轉成影像 pixel 座標。
- 若有 facial transformation matrix，先以該幀估計的 roll 對 ROI polygon 做旋轉補償。
- 用 ROI 點建立凸多邊形 mask。
- mask 面積小於 30 pixel 時視為無效。
- 對整張 BGR 影像做 5x5 Gaussian blur。
- 用 mask 取平均色彩。
- 回傳 (R, G, B)，注意 OpenCV 原影像是 BGR，因此程式會把 cv2.mean 的結果轉成 RGB 順序。

多個 ROI 不再固定平均，而是依頭部姿態動態分配權重：

- forehead：基礎權重最高，正面視角時優先。
- 左右 cheek：依 yaw 偏向自動重新分配，較朝向鏡頭的一側權重會提高。
- glabella：保留較低但穩定的輔助權重。

這樣可以降低側頭、歪頭、局部遮擋時單一 ROI 拉偏訊號的風險。若沒有任何有效 ROI，該幀不會推入 rPPG buffer。

系統也會對每個 ROI 做遮擋或異常色彩降權：

- ROI 平均亮度過暗或過亮時，代表可能受陰影、曝光或反光影響。
- 色度過低時，代表該區域可能缺乏有效皮膚顏色變化。
- 色度過高時，代表可能有強反光、飽和色或非皮膚遮擋。
- 遮擋分數會轉成 ROI 權重折扣，而不是直接讓整張臉失效。

目前 transformation matrix 的幾何補償採保守策略：已落地 roll 補償與姿態導向權重，尚未導入完整 3D mesh warp。這樣做的原因是完整 warp 會改變 ROI 取樣分布與 rPPG 數值行為，仍需要固定資料集回放驗證後再擴充。

5. POS 脈搏波演算法

本專案使用 POS（Plane-Orthogonal-to-Skin）類型演算法，程式位於 pos_pulse(r, g, b)。

輸入是 ROI 平均 RGB 時間序列。首先對每個色彩通道做時間標準化：

r_norm = (r - mean(r)) / (std(r) + 1e-8)

g_norm = (g - mean(g)) / (std(g) + 1e-8)

b_norm = (b - mean(b)) / (std(b) + 1e-8)

接著建立兩個投影分量：

x = 3R - 2G

y = 1.5R + G - 1.5B

用標準差比例估計混合係數：

alpha = std(x) / (std(y) + 1e-8)

最後輸出：

pulse = x - alpha * y

解釋：POS 的概念是把 RGB 皮膚顏色變化投影到與膚色方向正交的平面，盡量削弱光照與膚色基線的影響，保留與血液容積變化相關的週期成分。本專 alpha 動態縮放，讓 x/y 兩個投影分量在不同光照與膚色下能維持相對平衡。

6. 時間窗、取樣率與暖機

RppgEngine 預設：

- buffer_sec=12.0
- target_fps=30.0

每次 compute 會先刪除超過 12 秒的舊樣本。取樣率 fs 不是固定使用目標 FPS，而是用目前 buffer 的時間戳估算：

dt = (t_last - t_first) / (n - 1)

fs = clip(1 / dt, 8, 90)

暖機門檻：

- 至少需要 max(6 秒, buffer_sec * 0.5) 的樣本才視為暖機完成。
- 若樣本少於約 4 秒，僅回傳 warmup、亮度與 LOW confidence，不計算 BPM/HRV。

這樣做可以避免剛啟動時用太短波形估算頻譜，降低初期 BPM 跳動。

7. Butterworth Band-pass Filter

濾波函式為 butter_bandpass(sig, fs, low=0.65, high=3.7, order=3)。

目前 rPPG 主流程呼叫：

butter_bandpass(raw, fs, 0.65, 3.7)

頻率範圍換算成 BPM 約為：

- 0.65 Hz = 39 BPM
- 3.7 Hz = 222 BPM

設計目的：

- 去除低頻漂移，例如整體亮度變化、緩慢姿勢變化。
- 去除高頻雜訊，例如感測器雜訊、細微運動、壓縮造成的閃爍。
- 保留常見心率範圍。

程式優先使用 butter(..., output="sos") 搭配
sosfiltfilt。SOS 結構在數值上比傳統 b/a filter 更穩定。若 SOS
失敗，才退回 butter + filtfilt。filtfilt 與 sosfiltfilt
都是前後向零相位濾波，可避免相位延遲影響 peak 時間。

8. Welch PSD 與 BPM

BPM 由 welch_peak_hz 計算：

- 使用 Welch 方法估計濾波波形的 PSD。
- nperseg = min(n, max(64, int(fs * 5)))，約使用最多 5
  秒 segment。
- overlap 為 nperseg * 3 // 4。
- nfft=max(512, nperseg) 增加頻率解析度。
- window 使用 blackmanharris。
- 只在 0.7 Hz 到 3.5 Hz 搜尋最大功率峰。
- BPM = peak_freq_hz * 60。

搜尋頻段 0.7 到 3.5 Hz 約等於 42 到 210 BPM。這比 band-pass
略窄，避免濾波邊界附近的能量成為主峰。

9. SQI 訊號品質指標

SQI 由 compute_sqi 計算：

- 對濾波後波形做 Welch PSD。
- 計算總頻譜功率 total = sum(pxx)。
- 計算生理可能頻段 0.6 Hz 到 4.0 Hz 的功率 inband。
- SQI = clip(inband / total, 0, 1)。

意義：

- SQI 越高，代表波形能量越集中在合理心率頻段。
- SQI 低通常表示光照變化、臉部移動、ROI 受遮擋或影像品質不足。
- 融合分數中，若 SQI 低於可靠門檻，rPPG 子分數會被折扣。

9.1 統一品質分數

目前系統另外建立 quality_score，把以下因素整合成 0 到 1 的工程品質分數：

- 臉部姿態品質：roll、yaw、pitch 是否偏離穩定視角。
- 臉部覆蓋品質：face area 是否足夠大。

- 臉部動作品質：face center 的 motion proxy 是否過大。
- ROI 可用率：四個 ROI 中有幾個有效。
- 訊號品質：SQI 與 1 - ibi_artifact_ratio。

quality_score 的用途：

- GUI 直接顯示目前可用性。
- Web metrics、Web dashboard live/playback 與 SQLite 錄製會保留。
- 融合分數中，rPPG pillar 會再乘上這個品質權重，避免低品質影像把整體疲勞分數推高。

9.2 MediaPipe ROI 幾何穩定度

目前 rPPG ROI 取樣已加入 MediaPipe 幾何穩定度診斷：

- landmark_jitter_norm：追蹤臉部輪廓、ROI 與關鍵點的 frame-to-frame 平均位移，並以臉部對角線正規化。
- roi_area_stability：追蹤四個 ROI polygon 面積總和的短窗變化，面積快速跳動時降低品質分數。
- roi_smoothing_applied：表示該幀是否套用 ROI polygon EMA 平滑。
- canonical_roi_warp_used：表示是否套用實驗性的姿態導向 ROI 幾何正規化。

ROI polygon EMA 只作用在 rPPG ROI 幾何，不回寫 MediaPipe landmark，也不餵給疲勞眨眼、saccade 或 microsleep 等快速事件判定，避免低通濾波鈍化行為訊號。

canonical_roi_warp_enabled
預設關閉。此功能只作為回放實驗開關，啟用後可能改變 ROI 取樣分布與 rPPG 數值行為，必須用固定錄影與 profile sweep 比較後再評估是否採用。

10. Peak Detection 與 IBI

IBI（inter-beat interval）由 ibi_from_peaks 計算：

- 對濾波後 pulse 找 peak。
- 最短 peak distance 為 fs * 0.35，約限制最高 171 BPM。
- prominence 門檻為 std(sig) * 0.35 + 1e-6。
- 若 peak 少於 2 個，無法形成 IBI。
- IBI = diff(peaks) / fs * 1000，單位為 ms。

這裡的 IBI 是從 rPPG waveform 估計，不等同 ECG R-R interval，因此 HRV 指標只應視為短窗、非醫療級估測。

11. IBI 清理與 Artifact Ratio

validate_ibi_intervals 會移除不合理或局部不一致的 IBI：

- 移除 NaN/inf。
- 保留 300 到 2000 ms，約等於 30 到 200 BPM。

- 若樣本足夠，使用 5 點局部 median。
- 計算每個 IBI 與局部 median 的差值。
- 以 median(delta) + 5.2 * IQR(delta) 與 20% local_median 建立 adaptive threshold。
- 再做第二輪 5 點局部 median，保留與局部 median 差距不超過 30% 的 interval。

ibi_artifact_ratio 計算被清掉的比例：

artifact_ratio = 1 - clean_count / raw_count

若沒有 raw IBI，artifact ratio 回傳 1.0，代表目前無可用 IBI。

12. HRV 指標

12.1 RMSSD

hrv_time_domain 中的 RMSSD：

RMSSD = sqrt(mean(diff(IBI)^2))

RMSSD 反映相鄰心搏間期的短期變化。一般來說，短期 RMSSD 較低可能與壓力或交感活化相關，但 rPPG 估測容易受影像品質影響，因此本系統會搭配 SQI、artifact ratio 與 confidence 使用。

12.2 SDNN

SDNN = std(IBI, ddof=1)

SDNN 是 IBI 的標準差。短窗 SDNN 易受樣本長度影響，本系統主要作為輔助顯示。

12.3 LF/HF

lf_hf_from_tachogram 的流程：

- IBI ms 轉秒。
- 用 cumulative sum 建立 IBI 發生時間軸。
- 若總時間小於 4 秒，回傳 0。
- 用 interp1d 將不等間隔 IBI 序列內插到 4 Hz。
- 移除平均值。
- Welch PSD。
- LF = 0.04 到 0.15 Hz 功率總和。
- HF = 0.15 到 0.4 Hz 功率總和。
- LF/HF = LF / HF。

注意：標準 HRV 頻域分析通常需要更長時間窗。本專案使用 12 秒左右短窗，因此 LF/HF 只作趨勢與粗略壓力提示，不應作臨床判讀。

13. 壓力 Heuristic

stress_heuristic(lf_hf, rmssd) 將 LF/HF 與 RMSSD 轉為 0 到 1 的壓力機率。

若 LF/HF 或 RMSSD 不可用，回傳：

- stress_prob = 0.0
- stress_status = "calibrating"

有效時：

lf_term = clip((lf_hf - 1.2) / 2.5, 0, 1)

rm_term = clip((65 - rmssd) / 65, 0, 1)

prob = clip(0.45 * lf_term + 0.55 * rm_term, 0, 1)

文字狀態：

- < 0.35：輕鬆
- < 0.60：一般
- >= 0.60：緊張

設計邏輯：LF/HF 升高與 RMSSD
降低常被用作壓力或自律神經負荷的粗略線索，因此本系統把兩者合成壓力機率。RMSSD 權重 0.55，LF/HF 權重 0.45。

14. HRV Confidence

hrv_confidence(n_ibis, sqi, art_ratio) 依三個條件評估可信度：

- LOW：IBI 少於 6、SQI 小於 0.25，或 artifact ratio 大於 0.35。
- MEDIUM：IBI 少於 12、SQI 小於 0.45，或 artifact ratio 大於 0.15。
- HIGH：以上條件都未觸發。

這個 confidence 是工程品質門檻，用於提示使用者目前 HRV 是否足夠可信。

15. Waveform 與 PSD 預覽資料

GUI 與 Web dashboard 顯示用資料：

- waveform：取濾波波形最後最多 160 點。
- waveform 會做 z-score 標準化，方便顯示。
- PSD 預覽只保留 0.5 到 4.0 Hz。
- 若 PSD 點數超過 64，會等距抽樣，降低前端傳輸與繪圖負擔。

16. rPPG 與疲勞融合

融合邏輯在 gui/fatigue_scorer.py。rPPG 是綜合疲勞分數的 40%，行為疲勞是 60%。若其中一方不可用，另一方會承擔 100% 權重。

rPPG 子分數由三部分組成：

- stress_prob：60%
- BPM 偏離正常範圍：30%
- LF/HF 偏高：10%

BPM 正常範圍設定為 60 到 100。低於 60 或高於 100 時，依偏離量除以 40 映射到 0 到 1。

LF/HF 子分數：

lfhf_score = clip((lf_hf - 1.5) / (4.0 - 1.5), 0,
1)

SQI 折扣：

sqi_weight = clamp(sqi / 0.30)

rPPG pillar 最終會乘上 SQI 權重。也就是說，波形品質不足時，即使壓力或 BPM 子項偏高，rPPG 對綜合分數的影響也會下降。

綜合分數等級：

- 0.00 到 0.20：清醒
- 0.20 到 0.40：輕度疲勞
- 0.40 到 0.60：中度疲勞
- 0.60 到 0.80：高度疲勞
- 0.80 到 1.00：極度危險

17. Worker 運作模式

17.1 RppgWorker

只開啟 rPPG 時使用。流程：

- 檢查 MediaPipe、SciPy、OpenCV 是否可用。
- 確認或下載 Face Landmarker 模型。
- 建立 FaceLandmarkerOptions。
- 從 queue 取 frame。
- MediaPipe 偵測臉。
- 若有臉，抽 ROI、推入 RppgEngine、計算 metrics、輸出 overlay。
- 若無臉，reset engine，避免舊臉部訊號延續到新狀態。

17.2 CombinedWorker

同時開啟 rPPG 與行為疲勞時使用。它只建立一個 MediaPipe Face Landmarker，然後把同一份 landmark 給 rPPG 與 fatigue engine。優點：

- 避免同一幀跑兩次 MediaPipe。
- 降低 CPU/GPU 負載。
- rPPG ROI 與疲勞標註使用同一幀 landmark，時間一致。

目前此模式也會讀取 Face Landmarker 的 blendshape 與 transformation matrix 輸出，將表情訊號併入疲勞判定，並把同一幀的姿態補償 ROI 供 rPPG 使用。

17.3 CameraStreamWorker

攝影機 worker 負責：

- 開啟 camera backend。
- 設定解析度與 FPS。
- 等待第一幀，避免開啟成功但無影像。
- 用 sliding window 計算實際 FPS。
- 將 frame 送給 GUI 與 rPPG/fatigue queue。
- queue 滿時丟棄新 payload，避免分析 worker 落後造成延遲累積。

18. 錄製與 Web Dashboard

rPPG metrics 會被轉成兩種輸出：

18.1 Web Dashboard

_build_web_metrics 將 rPPG 欄位轉成前端使用的 stress_*
欄位，例如：

- stress_bpm
- stress_rmssd
- stress_sdnn
- stress_lf_hf
- stress_sqi
- stress_peak_freq
- stress_hrv_confidence
- stress_ibi_artifact_ratio
- stress_prob
- stress_waveform
- stress_psd_freqs
- stress_psd_values

18.2 SQLite

MetricsRecorder 寫入 metrics table。rPPG 相關欄位包括：

- rppg_face_detected
- bpm
- rmssd
- sdnn
- lf_hf
- sqi
- stress_prob
- stress_status
- hrv_confidence
- ibi_artifact_ratio

- peak_freq_hz
- frame_brightness
- warmup_progress
- rppg_quality_score
- rppg_pose_roll_deg
- rppg_pose_yaw_deg
- rppg_pose_pitch_deg
- rppg_roi_valid_count
- rppg_roi_weight_forehead
- rppg_roi_weight_l_cheek
- rppg_roi_weight_r_cheek
- rppg_roi_weight_glabella
- fatigue_quality_score
- fatigue_blend_calib_progress

舊版錄製資料庫在新版程式開啟時會以 ALTER TABLE 自動補欄。Web playback API 也會偵測欄位是否存在，舊 DB 缺少新診斷欄位時會回傳預設值，避免回放失敗。

18.3 回放校準摘要工具

scripts/replay_calibration_summary.py 可讀取 metrics_*.db 並輸出：

- 平均 rPPG quality score。
- 平均 fatigue quality score。
- 平均 SQI 與 BPM。
- 低品質筆數、高 artifact 筆數與 ROI 可用數不足筆數。
- forehead、左右 cheek、glabella 的平均 ROI 權重。

使用方式：

```
.venv/bin/python
scripts/replay_calibration_summary.py
recordings/metrics_YYYYMMDD_HHMMSS.db
```

18.4 MediaPipe profile sweep

scripts/mediapipe_profile_sweep.py 可用錄製影片重跑 MediaPipe Face Landmarker profiles，輸出：

- face detection rate。
- 平均 ROI 可用數。
- 平均 landmark jitter。
- 平均 ROI 面積穩定度。
- ROI smoothing 與 canonical ROI warp 的使用比例。

使用方式：

.venv/bin/python
scripts/mediapipe_profile_sweep.py
recordings/video_YYYYMMDD_HHMMSS.mp4 --csv
reports/mediapipe_profile_sweep.csv

此工具只比較 MediaPipe 偵測與 ROI 幾何穩定度，不重算或宣稱真實 BPM 精準度。若要評估心率誤差，仍需同步參考儀器 ground truth。

19. Demo 模組

rppg/demo_rppg_live.py 是獨立 demo，使用同樣概念：

- MediaPipe Face Landmarker。
- ROI 平均色彩。
- POS pulse。
- Butterworth band-pass。
- Welch peak BPM。
- peak-based IBI。
- RMSSD、SDNN、LF/HF、SQI、壓力 heuristic。

Demo 也支援：

- --list-cameras
- --pick
- --camera
- --backend auto/msmf/dshow/v4l2/any

它適合用於不啟動完整 GUI 時快速測試相機與 rPPG 訊號。

20. 目前限制與判讀注意事項

- rPPG 對光照、臉部移動、相機自動曝光、壓縮、膚色、遮擋與 FPS 穩定度敏感。
- 短時間窗 HRV 不等同標準臨床 HRV；LF/HF 尤其需要謹慎解讀。
- POS 演算法不是深度學習模型，優點是可解釋與輕量，缺點是對運動與光照變動較敏感。
- BPM 較可信時通常需要：臉部穩定、光照穩定、SQI 高、artifact ratio 低。
- stress_prob 是 heuristic，不是醫療診斷。
- face_landmarker.task 是臉部 landmark 模型，不是心率模型；心率由後續訊號處理演算法估算。
- blendshape 與 transformation matrix 已接入，但工程上仍以幾何 landmark 與訊號處理結果為主。

20.1 數值行為保護區

一般 MediaPipe 優化不應順手改動以下 rPPG 數值流程：

- POS 演算法公式、RGB 正規化方式與 RGB/BGR 通道順序。
- Butterworth band-pass 頻段、order 與零相位濾波流程。
- Welch PSD 的搜尋頻段、window、nperseg、overlap 與 nfft。

- buffer_sec、暖機門檻與取樣率 fs 估算方式。
- 四個 rPPG ROI 的 landmark index、mask 面積門檻、Gaussian blur 與平均色取樣方式。
- num_faces=1 的單人假設。

若需要更改上述項目，應以固定錄影資料與參考儀器 ground truth 建立獨立驗證任務。

21. MediaPipe 可優化項目、缺口與參數調整

目前 MediaPipe Face Landmarker 在本專案中的核心工作是：提供穩定 landmark、blendshape 與 transformation matrix，讓 rPPG 與疲勞引擎共用同一組臉部資訊。若要再提升準確度，優先順序如下。

21.1 已完成的優化

- output_face_blendshapes=True 已啟用，且已加入個人化 baseline 正規化。
- blendshape baseline 已支援跨 session 保存，儲存在 config/mediapipe_calibration.json。
- output_facial_transformation_matrixes=True 已啟用，且已用於 ROI roll 補償、姿態品質與 ROI 動態權重。
- 已加入 ROI 遮擋降權，降低反光、過暗、過曝或非皮膚色彩 ROI 的影響。
- MediaPipe 相關參數已外部化到 config/mediapipe_tuning.json。
- 已新增錄製回放品質摘要工具 scripts/replay_calibration_summary.py。
- Web dashboard live/playback 已顯示 quality、pose、ROI 權重與 blend calibration progress。
- SQLite schema 已新增診斷欄位，並支援舊資料庫 migration。
- num_faces=1 已固定，對單人 rPPG 是合理設定，也讓 smoothing 生效。
- quality_score 已建立，並已接到 GUI、Web metrics、SQLite 與融合分數。

21.2 目前仍可精修的部分

- transformation matrix 已落地為 roll 補償與姿態權重，完整 3D ROI warp 仍待固定資料集驗證後再導入。
- blendshape baseline 已可保存，但可再加入依光照與使用者狀態自動重估的長期漂移修正。
- 遮擋降權已完成第一版，可再加入眼鏡反光、口罩、手遮臉或側臉遮蔽的專用分類。
- 尚未將多臉追蹤拆成獨立流程；目前單人 rPPG 場景維持 num_faces=1。

21.3 參數調整建議

- min_face_detection_confidence

- 若誤抓背景或臉部誤檢偏多，可提高到 0.6 到 0.75。 - 若在低光、遠距離或小臉場景常常抓不到臉，可降到 0.4 到 0.5。

• min_face_presence_confidence

- 若 landmark 飄動、臉部存在感不穩，可提高到 0.55 到 0.7。 -
若追蹤過度保守、容易斷臉，可降到 0.4 到 0.5。

• min_tracking_confidence

- 若轉頭或快速移動時追蹤容易跳掉，可提高到 0.6 到 0.8。 -
若想降低短暫抖動造成的丟臉，可維持預設或略降到 0.4 到 0.5。

• output_face_blendshapes

- 若要維持目前眼、眉、嘴的輔助判定，應保持 True。 -
若只想追求最低推論成本，可關掉，但會失去表情輔助訊號。

• output_facial_transformation_matrixes

- 若後續要做 ROI 對齊、姿態補償或更穩定的頭部角度修正，應保持 True。 - 若只做最小化
landmark 擷取且不打算做幾何補償，才考慮關掉。

• num_faces

- 單人 rPPG 與疲勞監測建議維持 1。 - 多人場景雖可提高值，但會增加計算量，也會使
smoothing 效果下降。

21.4 下一階段建議

• 以固定資料集驗證完整 3D ROI warp 是否真的優於目前 roll 補償與姿態權重。

• 將遮擋偵測從色彩/亮度 heuristic 擴充為眼鏡、口罩、手遮臉等情境分類。

• 依錄製回放摘要建立場景化 tuning profile，例如低光、車內、辦公室或夜間。

• 若要支援多人，另建多臉追蹤與主體選擇策略，不直接改動單人 rPPG 主流程。

22. 官方文件參考

• MediaPipe Face Landmarker Python
官方文件：https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/python

• OpenCV VideoCapture
官方文件：https://docs.opencv.org/

• SciPy butter
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html

• SciPy sosfiltfilt
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.sosfiltfilt.html

• SciPy filtfilt
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html

• SciPy welch
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html

• SciPy find_peaks
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html

• SciPy interp1d
官方文件：https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html